ArborIQ: Urban Tree Lifecycle Management with Image and Disease Similarity Search

Course: Database Management Systems, Dept. of ISE, RVCE Project type: Application-style DBMS project with spatial and vector extensions Stack in one line: PostgreSQL + PostGIS (spatial) + pgvector (similarity)

---

1. Project Idea

Cities and campuses manage thousands of trees, but the records are usually scattered across spreadsheets, paper registers, and photos on phones. When a tree shows signs of disease, an inspector has to recall similar past cases from memory.

ArborIQ is a database-driven system that manages the full lifecycle of every tree (planting, growth, inspections, diseases, treatments, work orders, removal) and adds an AI-powered similarity layer. Inspection photos and written notes are converted into embeddings and stored in the same database as the normal records. An inspector can upload a photo of a suspicious leaf or bark patch and instantly see the most similar past disease cases, along with which treatments worked.

The key idea is that one database answers questions that mix location, relational facts, and visual similarity, for example: "Show the 5 most similar past disease cases for this species within this ward."

2. Problem Statement

- Tree records are fragmented, so inspection history, treatments, and locations cannot be queried together.
- Disease identification depends on individual experience, which is slow and inconsistent.
- Maintenance is reactive: trees are treated after serious damage rather than flagged early.
- There is no way to search past cases by what the symptom looks like.

3. Objectives

1. Design a normalized (up to 3NF) schema covering trees, species, locations, inspections, diseases, treatments, and work orders.
2. Store tree locations as spatial data and support proximity and ward-level queries.
3. Store embeddings of inspection photos and notes in vector columns and support similarity search.
4. Support hybrid queries that combine spatial, relational, and vector conditions.
5. Automatically flag high-risk trees using triggers when a new inspection resembles known severe cases.

6. Provide role-based access, audit logging, and input validation.
7. Evaluate how well similarity search retrieves relevant past cases.

4. Users and Roles

Role	What they can do
Admin	Manage users, species, diseases, and system settings; view everything
Supervisor (arborist)	Review flagged trees, approve diagnoses, create and assign work orders
Field inspector	Add inspections and photos, view similar cases, update assigned work orders
Citizen reporter (optional)	Report a problem tree with a photo and location; view only the status of their own reports

5. Functional Modules

1. Tree registry: add and update trees with species, planting date, girth, height, and location
2. Inspection management: record condition, notes, and photos for each visit
3. Disease and treatment catalogue: reference data for diseases, symptoms, and treatments
4. Diagnosis and treatment tracking: link inspections to diagnoses and treatments, with outcomes
5. Work order management: create, assign, and close maintenance tasks
6. Similarity search: find similar past cases by photo or text
7. Risk alerts: trigger-based flagging of high-risk trees
8. Reports and dashboards: tree health by ward, overdue inspections, treatment effectiveness

6. DBMS Components Used

Component	How it appears in ArborIQ
Relational schema and normalization	Entities for trees, species, wards, inspections, diseases, treatments, work orders, users; designed up to 3NF
Keys and constraints	Primary and foreign keys, unique, check, and not-null constraints (for example, valid health scores and valid dates)
Spatial data (PostGIS)	Point locations for trees, ward boundaries as polygons, distance and containment queries
Vector data (pgvector)	Embedding columns for inspection images and notes, with similarity search
Indexing	B-tree for ordinary lookups, spatial index (GiST) for locations, vector index (HNSW or IVFFlat) for embeddings
Joins and aggregation	Multi-table joins, grouping, window functions for trends and rankings
Views and materialized views	Ward-level health dashboard, latest inspection per tree
Triggers	Auto-flag high-risk trees, maintain audit log, update the "last inspected" date
Stored procedures and functions	Create a work order from an alert, compute a tree's risk score, retrieve similar cases
Transactions (ACID)	A diagnosis, its treatment, and the work order are created together or not at all
Concurrency	Several inspectors updating records at the same time without conflicts
Security	Role-based access, row-level security by ward, audit trail
Backup and recovery	Scheduled backup and a documented restore procedure

7. Core Data Model (Described)

Main entities and key attributes

Entity	Key attributes
Species	species_id, common name, scientific name, typical lifespan, growth rate
Ward	ward_id, name, boundary (polygon)
Tree	tree_id, species_id, ward_id, location (point), planting date, height, girth, status (healthy, under treatment, removed)
User	user_id, name, role, ward assignment
Inspection	inspection_id, tree_id, inspector_id, date, health score, notes, notes embedding
Inspection image	image_id, inspection_id, file reference, body part (leaf, bark, trunk, root), image embedding
Disease	disease_id, name, category (fungal, bacterial, pest, nutrient), severity level
Species–disease susceptibility	species_id, disease_id (resolves a many-to-many relationship)
Diagnosis	diagnosis_id, inspection_id, disease_id, confidence source (manual or suggested), confirmed by, date
Treatment	treatment_id, name, method, typical duration, cost
Treatment record	diagnosis_id, treatment_id, start date, end date, outcome
Work order	order_id, tree_id, diagnosis_id, assigned_to, priority, status, due date
Alert	alert_id, tree_id, inspection_id, reason, similarity score, status
Audit log	log_id, user_id, action, table, record_id, timestamp

Key relationships

- A species has many trees; a ward has many trees.
- A tree has many inspections; an inspection has many images.
- An inspection can lead to zero or more diagnoses; a diagnosis links to one disease.
- A diagnosis can have many treatments (through the treatment record).
- Species and diseases are many-to-many through the susceptibility table.
- An alert is tied to a tree and the inspection that triggered it.

Normalization note for the report: Show an initial single-table design (a flat inspection sheet repeating species, ward, and disease details), then walk through 1NF, 2NF, and 3NF, removing partial and transitive dependencies to arrive at the tables above.

8. Vector Layer Design

- What is embedded: inspection photos (leaf, bark, trunk), and optionally inspection notes and disease descriptions.
- How: A pretrained image model (such as CLIP or a lightweight vision model) runs in the application layer and returns a fixed-length vector for each image. A text embedding model does the same for notes. The vectors are then stored in the database.
- Dimension: fixed per column (for example, 512 for the image model), and validated on insert.
- Distance measure: cosine distance.
- Index: an approximate nearest neighbour index on each vector column.
- What is stored in the database: vectors, and a reference to the image file (the image itself can live on disk or object storage).

Important honesty point: Similarity search does not diagnose. It retrieves visually similar past cases for a human to review. The final diagnosis is always confirmed by a supervisor.

9. Hybrid Queries (Sample Query Execution)

These are described in words so they can be written in SQL during implementation.

1. Nearby similar cases: For a newly uploaded photo, find the 5 most similar past inspection images, only from trees of the same species within a chosen distance of the current tree.
2. Ward-level disease search: Show the 5 most similar past disease cases for a given species in a given ward, along with each case's diagnosis and the treatment that resolved it.
3. Cluster of risk: List trees within 500 metres of a confirmed severe case whose latest inspection looks similar to that case and whose last treatment is more than a year old.
4. Treatment effectiveness: For each disease, show the share of treatments with a successful outcome, ranked by cost.
5. Overdue inspections: Trees not inspected in the last six months, grouped by ward and ordered by risk.
6. Lifecycle report: Average age at removal by species, and the most common disease at removal.

10. Triggers and Stored Logic

- Risk alert trigger: When an inspection image is added, compare its embedding with confirmed severe-disease cases of the same species. If similarity is above a calibrated

threshold, create an alert and raise the tree's risk level.
- Work order procedure: A supervisor confirms an alert, and a procedure creates the diagnosis, treatment plan, and work order in one transaction.
- Last-inspected trigger: Updates the tree's summary fields after each inspection.
- Audit trigger: Logs inserts, updates, and deletes on sensitive tables with the user and time.

Design note: Running a nearest-neighbour search inside a trigger can slow inserts. For the report, test this and describe the alternative of running the check in a queued background step.

11. Security, Validation, and Forms

Security

- Role-based access control with the roles in Section 4
- Row-level security so inspectors see only their assigned wards
- Hashed passwords, and no direct database access for application users
- An audit log for sensitive changes
- Citizen reporters limited to their own reports

Validation

- Image type and size limits, and rejection of corrupt files
- Embedding dimension check on insert
- Valid GPS coordinates, and a check that the point falls inside a known ward
- Date and range checks (planting date not in the future, health score within its scale)

Forms

- Tree registration form
- Inspection form with photo upload
- Similar-case results page with a map
- Diagnosis and treatment form
- Work order form
- Alert review screen
- Dashboard with ward-level health

12. Novelty and Motivation

Why not just use a disease-classification model?

A trained classifier gives a label, but:

- It needs many labelled examples for every disease, which a city or campus rarely has.
- It must be retrained whenever a new disease appears.
- It does not explain itself. A similarity search shows the actual past cases behind a suggestion, and the treatments and outcomes attached to them.

Case retrieval inside the database keeps the system explainable, works with small datasets, and improves automatically as new confirmed cases are added.

Why use a database for this instead of a separate AI tool?

The value is in combining things a standalone model or a standalone vector store cannot do together: location, species, tree age, treatment history, access control, and visual similarity in one query and one transaction.

Our contributions

- Lifecycle plus similarity in one schema. A complete, normalized tree-management database with vector search built in rather than bolted on.
- Spatial, relational, and vector filters in a single query, which is the hybrid pattern real systems struggle with.
- Similarity-driven risk alerts implemented as database triggers and procedures.
- Case-based, explainable suggestions that show past diagnoses and treatment outcomes together.
- A measured evaluation of retrieval quality and filter-versus-vector trade-offs on real tree images.

What we do not claim

- We do not claim to replace an arborist or provide automatic diagnosis.
- We do not claim a new embedding model or new indexing algorithm. We use existing tools and focus on database design and integration.

Short answer for the viva

"Trees are managed by place, species, age, and history, and diseases are recognised by sight. Normally those live in separate tools. We put all of them in one database so a single query can combine location, relational facts, and visual similarity, and so that alerts, access control, and treatment history work together. We use similarity to retrieve past cases for a human to confirm, not to diagnose."

13. Innovative Experiment (10 Marks)

Experiment A: Retrieval quality

- Build a labelled set of images across several disease categories.

- For each test image, retrieve the top k similar images and measure how often the retrieved cases share the correct disease (precision at k).
- Compare two or more embedding models and compare against a baseline that searches only the text notes.

Experiment B: Hybrid filtering

- Run the same similarity query with no filter, a species filter, a spatial filter, and combined filters.
- Compare latency and result quality, with and without the vector index.

Experiment C: Alert trigger quality

- On a held-out set, measure how often the trigger correctly flags severe cases (recall) and how often it raises false alarms (precision), at different thresholds.

Present the results in tables and a short discussion of the best threshold and model.

14. Implementation Idea

Database: PostgreSQL with the PostGIS and pgvector extensions Backend: Any REST framework the team is comfortable with (for example Python with FastAPI, or Node.js) Frontend: Web app with a map view (a standard open-source map library) and forms for the modules above Embeddings: An open-source image-text model run in the backend; vectors are stored in the database Storage: Image files on disk or object storage, with paths in the database

Data plan

- Images: Photograph trees on campus or in a small area, supplemented by a public plant-disease image dataset for variety. Label disease categories with help from a domain expert or reliable reference.
- Records: Generate realistic synthetic data for inspections, treatments, and work orders so the relational queries have enough volume.
- Locations: Use real coordinates for a small area and ward-style polygons drawn around it.

Build order

1. Schema, constraints, and sample data
2. Spatial columns and queries
3. Roles, row-level security, and audit logging
4. Application forms for core modules
5. Embedding pipeline and vector columns
6. Similarity search and hybrid queries
7. Triggers, procedures, and alerts

8. Dashboards and views
9. Experiments and evaluation

15. Mapping to the Course Rubric

Rubric item	What to submit
Introduction and sample query execution	Sections 1 and 9, with queries run live
Synopsis	Sections 1 to 3, condensed
Requirement specification	Section 16
ER diagram	Entities and relationships from Section 7
DFD levels 0, 1, 2	Level 0: users and system. Level 1: modules from Section 5. Level 2: inspection-to-alert flow and similarity search flow
Normalized tables up to 3NF	The tables in Section 7 with the normalization walkthrough
Design of forms, security and validation	Section 11
Innovative experiment	Section 13
Demo and final report	Live demo and the outline in Section 17

16. Requirements

Functional

- Register trees with species, ward, and map location
- Record inspections with photos and notes
- Search similar past cases by image or text
- Combine spatial, relational, and similarity filters in queries
- Raise and review risk alerts
- Create and track treatments and work orders
- Produce dashboards and reports
- Manage users, roles, and access

Non-functional

- Accuracy: retrieval should reach a target precision at k on the test set

- Performance: similarity queries return within a few seconds on the demo dataset
- Integrity: no orphan records; a diagnosis, treatment, and work order are saved atomically
- Security: every action checked against the user's role and ward
- Usability: simple forms and a clear map view for field use

17. Report Outline

1. Title page and certificate
2. Abstract
3. Introduction and motivation
4. Problem statement and objectives
5. Literature survey (tree management systems, spatial databases, vector search)
6. Requirement specification
7. System design: ER diagram, DFD levels 0 to 2, normalization to 3NF
8. Database design: tables, constraints, indexes, views, triggers, procedures
9. Implementation: embedding pipeline, hybrid queries, application modules
10. Security and validation
11. Testing and results
12. Experiments and evaluation
13. Limitations and future work
14. Conclusion
15. References

18. Risks and Mitigation

Risk	Mitigation
Too few real disease images	Combine own photos with a public dataset; keep disease categories few and clear
General embedding models may not separate similar diseases well	Compare models, report results honestly, and include a human confirmation step
Incorrect labels	Have labels checked by a domain expert or a reliable reference
Trigger slows inserts	Measure it; move the check to a background step if needed
Scope too large	Finish the relational core first; add the vector layer second; treat the citizen portal as optional
Privacy of photos and locations	Avoid capturing people in images; restrict citizen access to their own reports

19. Limitations and Future Work

- Similarity is a decision aid, not a diagnosis.
- Quality depends on the size and correctness of the labelled case base.
- Photos vary with lighting, angle, and season, which affects matching.
- Future work: a mobile field app, satellite or drone imagery, seasonal trend analysis, and fine-tuning an embedding model on tree-specific data.

20. References

- Silberschatz, Korth, and Sudarshan, Database System Concepts
- PostGIS documentation
- pgvector documentation
- Radford, A. et al., Learning Transferable Visual Models From Natural Language Supervision (CLIP), ICML 2021
- Hughes, D. and Salathé, M., An open access repository of images on plant health to enable the development of mobile disease diagnostics, arXiv, 2015